
UNIDAD 03

Variables y operadores

Programación en Computación · Ingeniería Electromecánica — 2.º año · UTN FR Reconquista
Longhi Pablo · Talijancic Iván

Qué vamos a ver

01 `let` y `const` — y por qué **no** usamos `var`

02 **Convenciones** de nombres

03 **Tipos** de datos y `typeof`

04 Operadores: **aritméticos** y de asignación

05 Operadores **relacionales** y **lógicos**

06 Por qué `'20' + 5` da `'205'`

La clase pasada usamos variables sin explicarlas. Hoy vemos **por qué** se escriben así.

PARTE 1

| Declarar variables

const: el valor no se reasigna

src/app.js

```
const nominalVoltage = 380  
  
nominalVoltage = 220
```

SALIDA

```
TypeError: Assignment to constant variable.
```

El programa **falla**. Y eso es exactamente lo que queremos.

let: el valor puede cambiar

src/app.js

```
let counter = 0  
counter = counter + 1  
counter += 1  
counter++  
  
console.log(counter)
```

SALIDA

3

Las tres formas hacen lo mismo. `+=` y `++` son atajos.

La regla de la cátedra

const por defecto.
let sólo si cambia.

Cuando lees **const**, ya sabés que ese valor **no se toca en todo el programa**.

ES UNA GARANTÍA DEL LENGUAJE: TE AHORRA RASTREAR EL CÓDIGO PARA VER SI ALGUIEN LO MODIFICÓ.

En la práctica casi todo es **const**. **let** aparece en contadores y acumuladores — la unidad 05.

PREGUNTA

¿Qué imprime esto?

```
console.log(nombre)  
var nombre = 'Ana'
```



var no avisa

× CON VAR

```
console.log(nombre)  
var nombre = 'Ana'
```

```
undefined
```

Sigue andando. El error aparece 20 líneas más abajo.

✓ CON CONST

```
console.log(nombre)  
const nombre = 'Ana'
```

```
ReferenceError: Cannot  
access 'nombre' before  
initialization
```

Falla acá, y te dice por qué.

Y se escapa de los bloques

src/app.js

```
{  
  const insideBlock = 'existo sólo acá adentro'  
  console.log(insideBlock)  
}  
  
console.log(insideBlock)
```

SALIDA

```
existo sólo acá adentro  
ReferenceError: insideBlock is not defined
```

Con `const` y `let` la variable vive sólo entre las llaves donde nació. `var` se escapa y produce colisiones de nombres.

Las tres, comparadas

	CONST	LET	VAR
¿Se reasigna?	✗	✓	✓
¿Respeto { } ?	✓	✓	✗
¿Avisa si la usás antes?	✓	✓	✗
¿La usamos?	por defecto	si cambia	no

`var` no está prohibido por el lenguaje: está prohibido **en esta cátedra**. Vas a verlo en tutoriales viejos; ahora sabés por qué no lo copiás.

PARTE 2

| Cómo se nombran

Tres convenciones

camelCase

Variables y funciones

`nominalVoltage`

`dailyHours`

PascalCase

Clases

`ElectricMotor`

no las usamos en el curso

UPPER_SNAKE

Constantes del problema

`NOMINAL_VOLTAGE`

`TOLERANCE`

Los **identificadores en inglés**; los textos que ve el usuario, **en español**.

El nombre es documentación

× ASÍ NO

```
const x = 380  
const y = 4.2  
  
console.log(x * y)
```

¿Qué es `x` ? Hay que leer todo el programa.

✓ ASÍ SÍ

```
const voltage = 380  
const current = 4.2  
  
console.log(voltage * current)
```

Se explica solo.

La prueba: si alguien lee **una sola línea**, ¿entiende qué es ese valor?

PARTE 3

| Tipos de datos

Cinco tipos

TIPO	QUÉ GUARDA	EJEMPLO
<code>number</code>	Números, con o sin decimales	<code>220</code> · <code>4.2</code>
<code>string</code>	Texto	<code>'M-14'</code>
<code>boolean</code>	Verdadero o falso	<code>true</code> · <code>false</code>
<code>undefined</code>	Declarada, sin valor asignado	
<code>null</code>	Vacío a propósito	

`220` y `4.2` son los dos `number` : **no hay un tipo entero y otro decimal.**

typeof: tu herramienta de diagnóstico

```
src/app.js
```

```
console.log(typeof 380)
console.log(typeof 'M-14')
console.log(typeof true)
console.log(typeof parseFloat('380'))
```

SALIDA

```
number
string
boolean
number
```

Cuando una cuenta da un resultado imposible, imprimí el `typeof` de lo que estás sumando. **Nueve de cada diez veces, algo que creías número es texto.**

undefined no es null

UNDEFINED

Nadie le asignó nada. Declaraste y no inicializaste.

```
let temperature
```

NULL

Vacío a propósito. Vos decidiste que acá no hay dato.

```
const noSensor = null
```

ERROR FRECUENTE

`typeof null` devuelve `'object'`, no `'null'`. Es un error del diseño original de JavaScript, de 1995, que nunca se corrigió para no romper la web entera. No hay nada que entender.

PARTE 4

| Operadores aritméticos

Seis operadores

OPERADOR	QUÉ HACE	10 Y 3
+	Suma	13
-	Resta	7
*	Multiplicación	30
/	División	3.333...
%	Resto de la división	1
**	Potencia	1000

Precedencia: primero `*` `/` `%`, después `+` `-`. Como en matemática, y **los paréntesis mandan**.

PREGUNTA

¿Para qué sirve el resto de una división?

$10 \% 3$ es 1. ¿Y con eso qué hago?



El resto sirve para dos cosas

src/app.js

```
const measurements = 30

console.log(measurements % 2 === 0)    // ¿es par?

const sensors = 17
console.log(sensors % 5)               // de 5 en 5, ¿cuántos sobran?
```

SALIDA

```
true
2
```

Un número es **par** si al dividirlo por 2 no sobra nada.

PARTE 5

| Comparar

Siempre devuelven **true** o **false**

src/app.js

```
const measured = 219.4
const nominal = 220

console.log(measured < nominal)
console.log(measured === nominal)
```

SALIDA

```
true
false
```

Un operador relacional **no decide nada**: sólo responde una pregunta.

QUIEN DECIDE ES EL **IF** DE LA CLASE QUE VIENE.

PREGUNTA

¿Es '5' igual a 5?

El texto '5' contra el número 5.



Depende de cuál uses

```
src/app.js
```

```
console.log('5' === 5)
console.log('5' == 5)
```

SALIDA

```
false
true
```

ERROR FRECUENTE

`==` **convierte los tipos por su cuenta** antes de comparar. Cómodo hasta que comparás algo que vino de un `prompt()` — que es texto — con un número, y el bug aparece tres líneas más abajo cuando intentás sumarlos.

Regla de la cátedra

Siempre `===` y `!==`

Comparan **valor y tipo**, sin conversiones a tus espaldas.

ERROR FRECUENTE

Usar `==` **se corrige como error grave**. Es lo que más se cobra en el parcial.



PARTE 6

| Combinar condiciones

Tres operadores

OPERADOR	NOMBRE	DA <code>TRUE</code> CUANDO...
<code>&&</code>	Y	las dos son verdaderas
<code> </code>	O	al menos una es verdadera
<code>!</code>	No	invierte: <code>!true</code> es <code>false</code>

Precedencia: `!` primero, después `&&`, después `||`. 🙌 **Poné paréntesis.**

&& – un caso real

src/app.js

```
const NOMINAL_VOLTAGE = 380
const TOLERANCE = 0.05
const measured = 372.5

const lowerLimit = NOMINAL_VOLTAGE * (1 - TOLERANCE)
const upperLimit = NOMINAL_VOLTAGE * (1 + TOLERANCE)

console.log(measured >= lowerLimit && measured <= upperLimit)
```

SALIDA

```
true
```

380 V con $\pm 5\%$. Las **dos** condiciones tienen que cumplirse $\rightarrow$ `&&`.

|| – alcanza con una

src/app.js

```
const hours = 4200
const temperature = 91

// A revisión si supera 10.000 h 0 si pasó los 85 °C
console.log(hours > 10000 || temperature > 85)
```

SALIDA

```
true
```

Las horas no llegan, pero la temperatura sí. Con una alcanza.

PARTE 7

| La trampa de los tipos

PREGUNTA

'20' + 5 da '205'.
¿Y '20' - 5?

Lo vimos la clase pasada con `prompt()`.



El + es el raro

src/app.js

```
console.log('20' + 5)
console.log('20' - 5)
console.log('20' * 5)
```

SALIDA

```
205
15
100
```

El `+` sirve para **dos cosas**: sumar números y pegar textos. Si un lado es texto, gana el pegado.

Los demás no tienen ese doble uso: convierten y calculan.

De ahí sale la regla de la clase pasada

Si el dato es un número, convértelo

src/app.js

```
const measured = parseFloat(prompt('Tensión [V]: '))
```

`prompt()` no está roto: devuelve `string`, y con `+` el texto gana.

ERROR FRECUENTE

`parseFloat('cuatro')` devuelve `NaN` — *Not a Number*. Es de tipo `number`, y **contagia toda la cuenta**. Si un resultado sale `NaN`, busca el `parseFloat` que recibió texto.

Todo junto

src/app.js

```
const NOMINAL_VOLTAGE = 380
const TOLERANCE = 0.05

const measured = parseFloat(prompt('Tensión medida [V]: '))
const lowerLimit = NOMINAL_VOLTAGE * (1 - TOLERANCE)
const upperLimit = NOMINAL_VOLTAGE * (1 + TOLERANCE)

console.log(`En rango: ${measured >= lowerLimit && measured <= upperLimit}`)
```

SALIDA

```
Tensión medida [V]: 372.5
En rango: true
```

Lo que hay que llevarse

01 `const` por defecto, `let` si cambia, `var` **nunca**.

02 `camelCase` en **inglés** para variables; `UPPER_SNAKE_CASE` para constantes.

03 `typeof` es la herramienta cuando algo no cierra.

04 **Siempre** `===` . Nunca `==` .

05 `&&` y `||` devuelven `true` / `false` : son la materia prima de la clase que viene.

PRÓXIMA CLASE • UNIDAD 04

Condicionales

`if` • `else if` • `else` • `switch-case`

Hoy imprimimos `true` y `false` . La próxima, deciden.